

Name : Sunny

CU240251760

Sec: B

Roll No : 58

Computer Vision

Labsheet – 7

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("video.mp4")

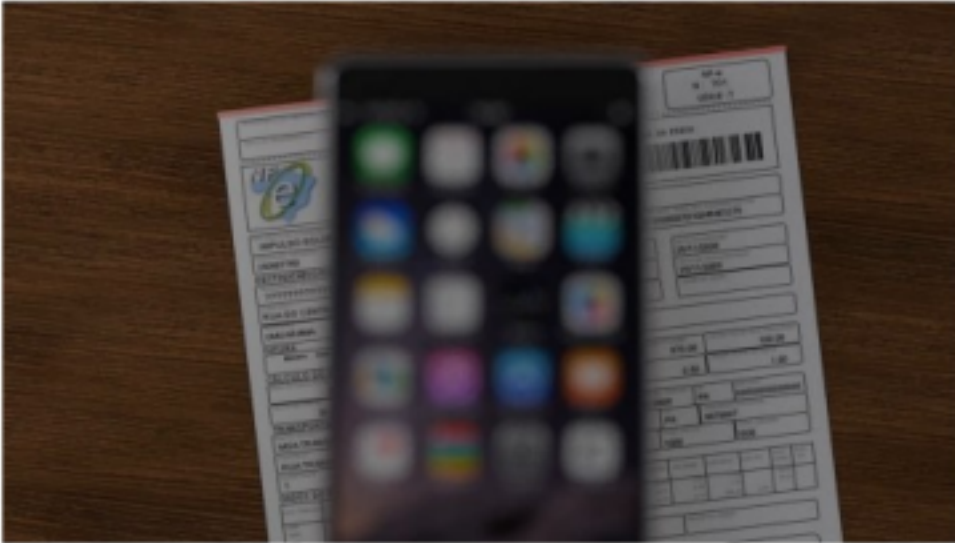
if not cap.isOpened():
    print("Error: Video not found")
else:
    print("Video loaded successfully")

Video loaded successfully

ret, old_frame = cap.read()

if not ret:
    print("Unable to read video")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

plt.imshow(cv2.cvtColor(old_frame, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params) print("Feature
points detected:", len(p0))

Feature points detected: 100

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

ret, frame = cap.read()
frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray, frame_gray, p0, None, **lk_params
)

good_new = p1[status == 1]
good_old = p0[status == 1]

print("Tracked points:", len(good_new))

Tracked points: 100
```

```

result = frame.copy()

for new, old in zip(good_new, good_old):
    x1, y1 = new.ravel()
    x2, y2 = old.ravel()

    cv2.arrowedLine(
        result,
        (int(x2), int(y2)),
        (int(x1), int(y1)),
        (0, 255, 0),
        2
    )
    cv2.circle(result, (int(x1), int(y1)), 3, (0, 0, 255), -1)

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()

```

Lucas-Kanade Sparse Optical Flow



```
flow = cv2.calcOpticalFlowFarneback(  
    old_gray,  
    frame_gray,  
    None,  
    0.5,
```

```
    3,  
    15,  
    3,  
    5,  
    1.2,  
    0  
)
```

```
print("Farneback Dense Optical Flow calculated")
```

Farneback Dense Optical Flow calculated

```
magnitude, angle = cv2.cartToPolar(  
    flow[..., 0],  
    flow[..., 1]  
)
```

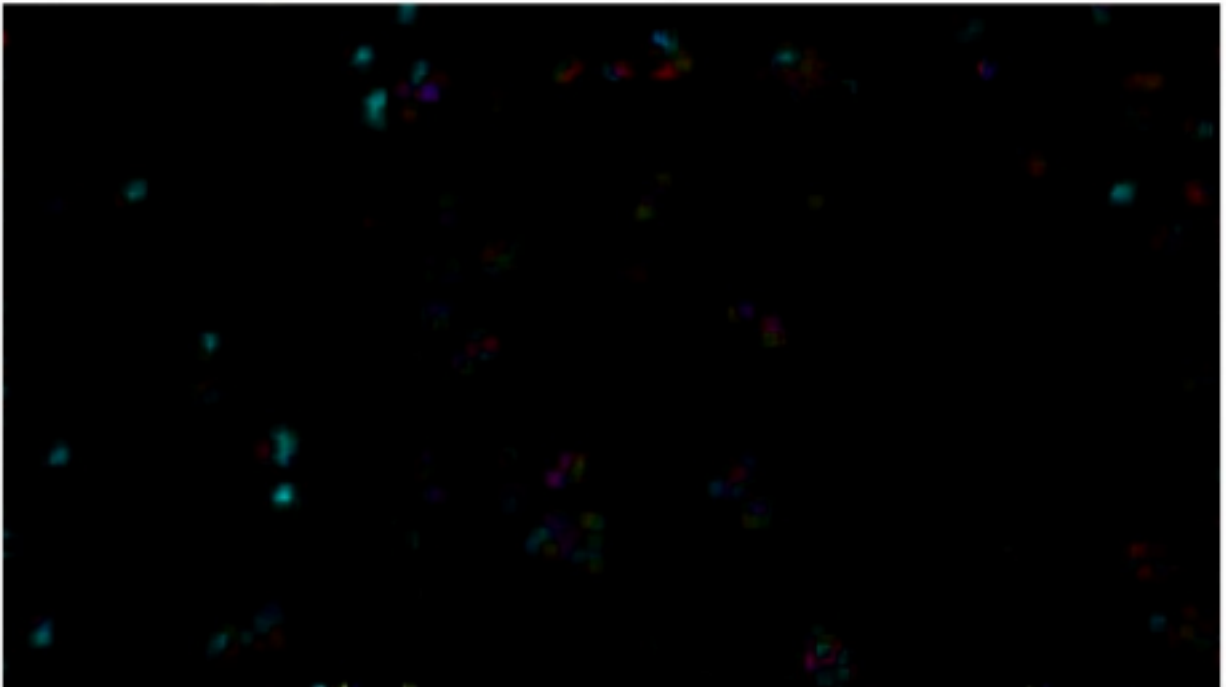
```
hsv = np.zeros_like(frame)  
hsv[..., 1] = 255
```

```
hsv[..., 0] = angle * 180 / np.pi / 2  
hsv[..., 2] = cv2.normalize(  
    magnitude,  
    None,  
    0,  
    255,  
    cv2.NORM_MINMAX  
)
```

```
dense_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)
```

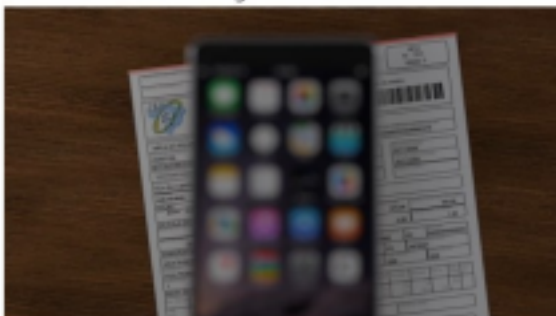
```
plt.figure(figsize=(10, 6))  
plt.imshow(dense_flow)  
plt.title("Farneback Dense Optical Flow")  
plt.axis("off")  
plt.show()
```

Farneback Dense Optical Flow



```
plt.figure(figsize=(12, 5))  
  
plt.subplot(1, 2, 1)  
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))  
plt.title("Original Frame")  
plt.axis("off")  
  
plt.subplot(1, 2, 2)  
plt.imshow(dense_flow)  
plt.title("Dense Optical Flow")  
plt.axis("off")  
  
plt.show()
```

Original Frame



Dense Optical Flow



```
print("Optical Flow Comparison")
print("-----")
print("Lucas-Kanade : Sparse, tracks selected feature points") print("Farneback : Dense,
estimates motion for almost every pixel") print("Lucas-Kanade : Faster and suitable for object
tracking") print("Farneback : More detailed but computationally expensive")
```

Optical Flow Comparison

Lucas-Kanade : Sparse, tracks selected feature points

Farneback : Dense, estimates motion for almost every pixel Lucas-Kanade :

Faster and suitable for object tracking

Farneback : More detailed but computationally expensive

```
cap.release()
```

```
print("Video processing completed successfully.");
```

Video processing completed successfully.

THEORY QUESTION

Q1. What is Optical Flow? How is it used in computer vision?

Optical Flow is a computer vision technique used to estimate the motion of objects between two consecutive video frames. It determines the direction and approximate speed of pixel movement. Optical Flow is useful in object tracking, video surveillance, autonomous driving, robotics, gesture recognition, and sports analytics.

Q2. Explain the working principle of Lucas-Kanade Optical Flow algorithm.

Lucas-Kanade is a Sparse Optical Flow algorithm that estimates the movement of selected feature points between consecutive frames. It assumes that the brightness of an object remains approximately constant and nearby pixels have similar motion. Image gradients are used to calculate the displacement of these feature points. It is commonly used for tracking objects and important features.

Q3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Dense Optical Flow calculates the motion for almost every pixel in an image, while Sparse Optical Flow calculates motion only for selected feature points. Dense Optical Flow gives more complete information about the movement in a scene but requires more computation. Farneback is an example of a Dense Optical Flow algorithm.

Q4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Lucas-Kanade is a Sparse Optical Flow method that tracks selected feature points, making it faster and computationally efficient. Farneback is a Dense Optical Flow method that estimates motion for almost every pixel. Lucas-Kanade is suitable for feature and object tracking, whereas Farneback is useful when detailed motion information across the complete image is required.

Q5. What assumptions are made while computing optical flow?

Optical Flow computation is based on some important assumptions. First, the brightness of an object remains constant between consecutive frames. Second, the motion between frames is assumed to be relatively small. Third, nearby pixels are assumed to have similar motion. These assumptions make it possible to estimate the movement of objects from image sequences.

Q6. What factors can affect the accuracy of optical flow estimation?

Several factors can affect Optical Flow accuracy, such as object speed, lighting conditions, camera movement, image noise, shadows, occlusion, and motion blur. Large movements between frames can also make tracking difficult. Changes in lighting and poor-quality video may result in incorrect motion vectors.

Q7. Mention five real-world applications of optical flow in computer vision.

Five important applications of Optical Flow are:

1. Autonomous driving – detecting movement of vehicles and pedestrians.
2. Video surveillance – detecting and tracking moving objects.
3. Robotics – understanding movement and navigation.
4. Gesture recognition – identifying hand and body movements.
5. Sports analytics – analyzing player and object movement.

It is also used in augmented reality and other dynamic scene applications.

Q8. Why are grayscale images generally used for optical flow computation?

Grayscale images are generally used because Optical Flow mainly depends on brightness and intensity changes between frames. Converting a color image into grayscale reduces the amount of data that needs to be processed. It also makes gradient calculations simpler and improves computational efficiency while estimating motion.

Q9. What are the limitations of optical flow algorithms in real-world environments?

Optical Flow algorithms may produce inaccurate results under difficult conditions such as fast object movement, sudden lighting changes, camera motion, shadows, occlusion, and motion blur. Textureless regions are also difficult to track because they contain fewer useful features. Therefore, real-world environmental conditions can reduce the accuracy and reliability of Optical Flow estimation.

Q10. How does optical flow contribute to autonomous driving, video surveillance, and action recognition?

Optical Flow provides information about how objects and pixels move over time. In autonomous driving, it helps detect moving vehicles, pedestrians, and changes in the scene. In video surveillance, it helps track objects and identify motion. In action recognition, motion patterns between video frames help identify activities such as walking, running, or waving.